

```
def getBalance(self, password):
    if password != self.password:
        print('Sorry, incorrect password')
        return None
    return self.balance

# Added for debugging
def show(self):
    print('        Name:', self.name)
    print('        Balance:', self.balance)
    print('        Password:', self.password)
    print()
```

Listing 1-7: First example of a class in Python

For now, take a look at the functions and see how they're similar to our earlier procedural programming examples. The functions have the same names as in the earlier code—`show()`, `getBalance()`, `deposit()`, and `withdraw()`—but you'll also see the word `self` (or `self.`) peppered throughout this code. You'll learn what this means in the next chapters.

Summary

This chapter started with a procedural implementation of the code for a card game called Higher or Lower. In Chapter 12, I will show you how to make an object-oriented version of the game with a graphical user interface.

I next introduced the problem of simulating a bank with one, then several bank accounts. I discussed several different ways to use procedural programming to implement the simulation and described some of the problems that this approach creates. Finally, I gave a first glimpse of what the code describing a bank account would look like if it were written using a class.